

The Conditional Variational Autoencoder

Gabriel Agostine
University of Colorado Boulder
gabriel.agostine@colorado.edu

Abstract

Within the world of Autonomous Systems, algorithms (or methods as called in the general sense) often need to keep track of some manner of state. This state may be a physical or digital state (or perhaps even a mix of the two), it must be accurately tracked regardless. Tracking this state often requires completing mathematical computations upon it, which can be time consuming or infeasible to the overall algorithm depending on its size. This naturally leads to a desire for a more compact—yet still equally representative—state that we can replace the true state with for these computations; we call this the latent state.

While compressing the true state into the latent state does allow for more computationally efficient mathematical computations, it also holds information about the state itself. By conditioning our state upon additional information, we can use this conditioned latent state for a variety of purposes including state estimation, compression and even reconstruction. The latter is the purpose of the Variational Autoencoder (VAE) and its younger sibling: the Conditional VAE or CVAE.

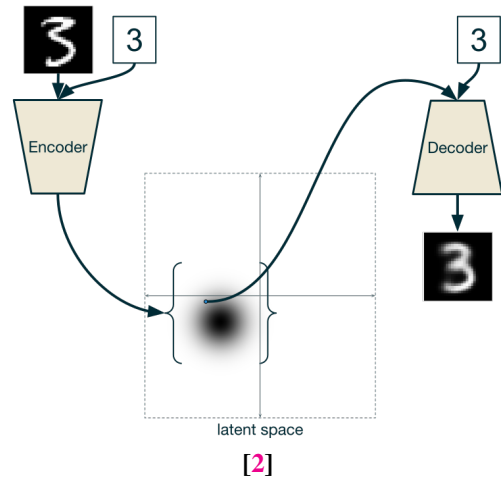
Introduction

The Variational Autoencoder (VAE) is a dual Multilayer Perceptron (MLP) or Neural Net (NN) that uses an encoder and decoder for the primary purpose of image reconstruction. The Conditional Variational Autoencoder (CVAE) does much the same but is further conditioned with additional information. Primarily, these two structures are used for image reconstruction; its use is not limited to images however and may include state estimation or prediction. These CVAEs (specifically the encoder architecture) are commonly used to compress the system's state into a more handleable size for the algorithm. How exactly the CVAE itself works, despite being a fundamental concept, is not always described.

This paper aims to reconstruct the findings of the original authors (Sohn et al. 2015, [1]) to further both the knowledge of the author and readers. Our CVAE implementation was implemented in Python 3.14 using minimal external libraries. We compare our custom implementation of the CVAE architecture against the same baselines the original authors do: the MNIST Digits and Labeled Faces in the Wild (LFW) datasets. Our custom implementation is also directly compared to the official PyTorch implementation to confirm the accuracy of our results.

Background & Related Work

The CVAE is a dual MLP or NN that takes in the system state and compresses it into a latent state space through a forward pass through a series of hidden layers. This process is carried out through the appropriately named encoder network (or just encoder). The decoder takes a point in the latent state space and reconstructs the full state via another forward pass through a series of hidden layers.



Both the encoder and decoder are (as mentioned prior) MLPs or NNs, to which the typical forward and backwards pass operations are applied with an Adam optimizer to update the model parameters.

Problem Formulation

Understanding the latent state space

Given a system state of high dimensionality, we can compress this state into a compact state representation that ideally holds all the relevant information in a more computationally operable manner. An example of this idea is seen in the numeral system itself. The base 2 numeral system, binary, is a system that has only two possible values: 0 and 1. The numeral system humans are familiar with is decimal, which has 10 possible numbers: 0 - 9. The number 20 (in decimal) is represented as 00010100 in binary. Even more compact numeral systems exist as well, such

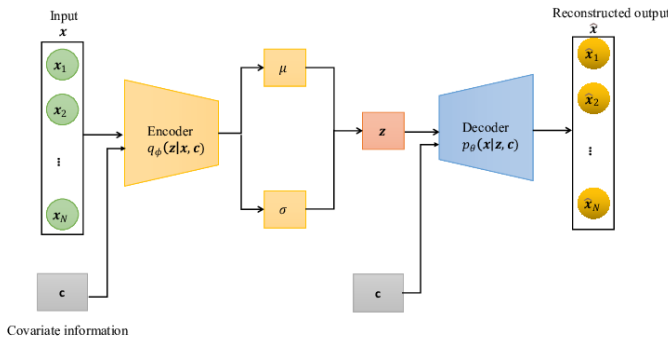
as Base32. In Base32 (Josefsson 2006, [3]) the decimal number 20 is represented as U (uppercase letter u). In each case, we use less and less "symbols" (be it actual numbers of letters) to encode the information of our "state" (the actual number we care about, regardless of the numeral system it is represented in). All this goes to show is that the concept of compressing information while maintaining its representation or purpose is not far-flung, such is what the encoder does with our input state.

The latent space (when the encoder network is properly trained) is a compact representation that encodes the most relevant features of the original input (in this case, the state). In NNs, there is no explicit, interpretable mapping between the input and the latent variables; the weights and biases are learned automatically during training and are unitless. Essentially, the latent space is an abstract feature space where the model stores information necessary for downstream tasks, capturing patterns in the original state.

It should be noted that in the base VAE, the latent space encodes information about the state. However, this is different in the CVAE, where the state is conditioned with further information. In this case, the latent space encodes low-level information about the state while the condition itself holds the high-level information about the state. Looking back at our numeral system example, the condition holds the information about "what" number we are encoding while the latent space would encode information about "how" the number looks (like font, size, etc.).

The encoder network

The latent space itself is determined by the encoder output. The encoder, as mentioned prior, takes in the conditioned state; a concatenation of the raw (sometimes resized) state x and condition vector (usually a one-hot vector) c . The encoder then maps it into a probability distribution in the latent space via the approximate posterior $q_\phi(z|x, c)$; not a point in the latent space however. This probability distribution is modeled by a non-standard Gaussian with parameters $\mu(x, c)$ and $\log(\sigma^2(x, c))$; the mean and logarithmic variance. This mean and log-variance are generated via a multi-headed final layer in the encoder architecture.



[4]

Once we have this non-standard Gaussian, we can sample a point

in the latent state space to get our latent state z from:

$$z \sim \mathcal{N}(\mu(x, c), \sigma(x, c)^2)$$

Small problem, big consequences

Those with a keen eye will realize a problem with the idea of sampling a random point in the probability distribution of the latent space. When training NNs, we must update the weights and biases of the network using the concept of backpropagation. This involves determining how much each specific weight contributes to the overall output. After determining the loss (or difference between the original input and generated output) we can find the gradients that model how changing each weight affects the loss.

Sampling in general is non-differentiable however, meaning we cannot propagate gradients through this "random variable." This presents a rather large problem from a small issue: how are we going to train these encoder and decoder networks if we cannot successfully backpropagate?

The Reparameterization Trick (originally suggested by Kingma et al. 2013, [5]) suggests modeling the latent state z as a deterministic function defined by:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Doing so makes it such that the random variable (ϵ) used to sample from probability distribution of the latent space from the encoder is now independent from the NN parameters. Essentially, since we don't need to differentiate with respect to ϵ , we can now properly backpropagate our gradients through sampling.

One more (small) issue arises from this trick: the distribution from which ϵ is sampled must be a Normal distribution with mean 0 and variance 1. This requires that the probability distribution of the latent space from the encoder network must model a Normal distribution with mean 0 and variance 1 as well. This can be dealt with by including an additional loss term, which will be discussed a bit later.

The decoder network

We now have our latent state z which may be used for state estimation, compression and even reconstruction. In the case of most VAEs, we often wish to reconstruct our state to see how similar it is to our input state. Such is how we calculate the resultant loss from a forward pass through the encoder and decoder networks.

To reconstruct our input, we condition the latent state z with our condition c once again and pass it through our decoder MLP or NN. The decoder maps our latent state into a reconstructed state $\hat{x}$ via the approximate posterior $p_\theta(\hat{x}|z, c)$.

Network loss

We may now calculate the loss as a result of a forward pass through the encoder-decoder network. The loss based off of two important terms:

- Reconstruction loss: quantifies the quality of the reconstructed state compared to the true state ($|x - \hat{x}|$)
- Latent Regularization loss: quantifies how well the non-standard Gaussian's averaged over training approximate a Gaussian normal

The first term is often called the Reconstruction loss. It simply quantifies how well the encoder captures the relevant information from the input state as well as how well the decoder reconstructs the input state. This usually means finding the Binary Cross-Entropy (BCE) between each element of the state and state reconstruction:

$$\text{BCE}(x, \hat{x}) = - \sum_i [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)]$$

where i is an iterator for each element of x and $\hat{x}$.

The BCE loss term helps drive the decoder networks to produce more accurate reconstructions given the latent space and the encoder to more accurately draw relevant information from the true state.

The second term is called Latent Regularization, but is more commonly known as the Kullback-Leibler (KL) Divergence. The KL Divergence is a measure of how much an approximating probability distribution is different from a true probability distribution (Kullback et al. 1951, [6]). As mentioned prior, we needed to add an additional loss term to ensure that the probability distribution of the latent space from the encoder network must model a Normal distribution with mean 0 and variance 1.

$$\text{KL}(\mu(x, c), \sigma(x, c)^2) = \left(\mathcal{N}(\mu(x, c), \sigma(x, c)^2) \parallel \mathcal{N}(0, I) \right)$$

The KL Divergence quantifies how different this probability distribution from the encoder network approximates a Normal distribution. This loss helps the encoder network learn to output a mean and log-variance that approximate a Gaussian Normal. As this term tends to 0, our assumption that the decoder receives a point sampled from a Gaussian Normal through the random variable ϵ is correct.

Thus our loss is as such:

$$\mathcal{L}(x, \hat{x}) = \text{BCE}(x, \hat{x}) + \text{KL}(\mu(x, c), \sigma(x, c)^2)$$

Solution Approach

A custom implementation of the previously described CVAE was created for the testing and confirmation of the original authors findings (Sohn et al. 2015, [1]). It was implemented in

Python 3.14 without the use of any official PyTorch or TensorFlow libraries. The standard Adam optimizer was also manually implemented with the defined learning rate α used to update the model parameters using their gradients.

Dataset	h_{dim}	l_{dim}	α	B	Number of Epochs
LFW	256	128	0.01	1024	2000
MNIST Digits*	128	64	0.01	1024	1000

h_{dim} and l_{dim} are the size of the hidden and latent space layers of both the encoder and decoder networks.

We trained on the Labeled Faces in the Wild (LFW) and a toy version of the MNIST Digits datasets.

The LFW dataset is a collection of JPEG images of famous people collected over the internet. The image itself is the "state" and the "condition" is the unique ID number of the individual. These images are of size 250x250 pixels and were greyscaled before being passed into the encoder network.

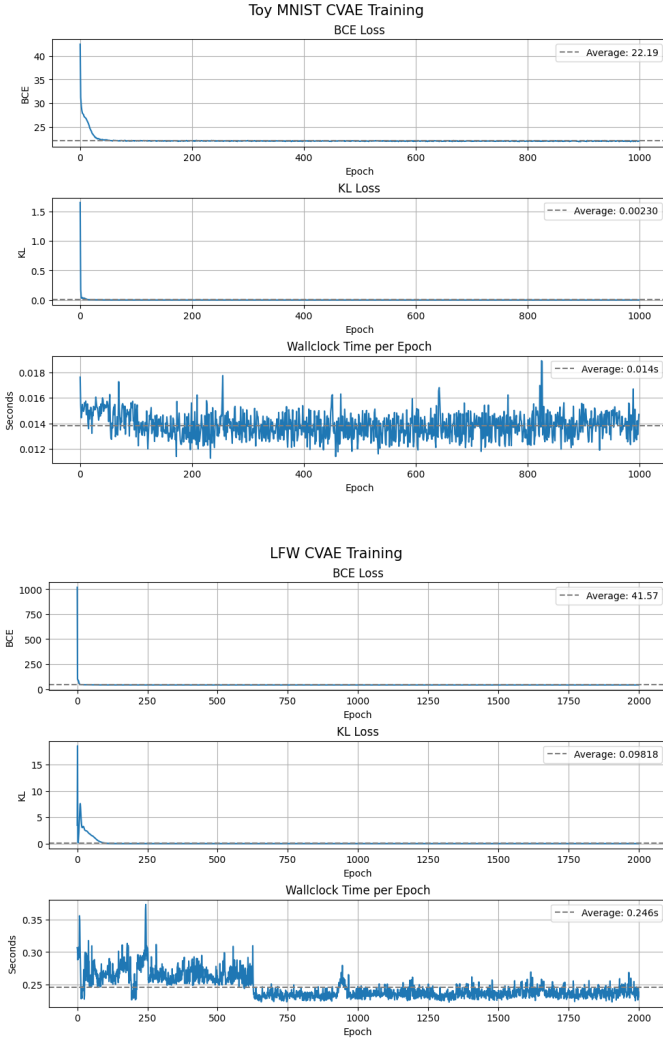
The MNIST Digits dataset is a collection of images of handwritten numbers. Again, the image itself is the "state" while the "condition" is the number that the image is of. The Toy dataset has size 8x8 pixels and is already greyscaled prior.

The Binary Cross-Entropy (BCE), Kullback-Leibler (KL) Divergence and wall-clock time were recorded after each epoch to monitor our implementations. We also used the official PyTorch implementation of a CVAE to confirm the validity of our results; the results seen in the following section are of our implementation however.

*The toy version of this dataset was chosen as Python 3.14 is still in its "bugfix" phase; the owner of the MNIST Digits dataset has not updated their dataset for Python 3.14. This does not take away from the effectiveness of the reconstructions however, the images are just of a smaller size.

Results

Model Losses



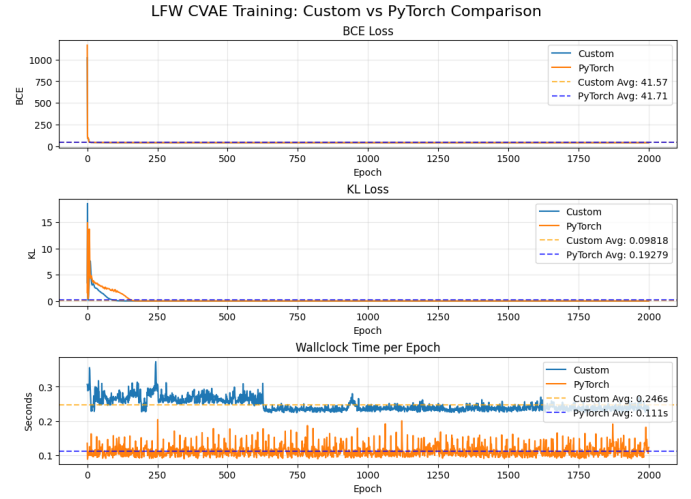
Dataset	Average BCE	Average KL Divergence	Average Epoch Wall-clock (s)
LFW	40.8	0.0001	0.25
MNIST Digits	22.1	0.0001	0.014

The BCE loss term exhibited exponential decay for both datasets at comparable rates, indicating that the quality of the reconstructions generally improved with increasing training epochs. The loss consistently stabilized at a positive value, suggesting a practical limit on reconstruction quality determined by the model architecture or training method.

The KL divergence loss displayed minor oscillations during the initial stages of training, as anticipated. In most cases, it decayed

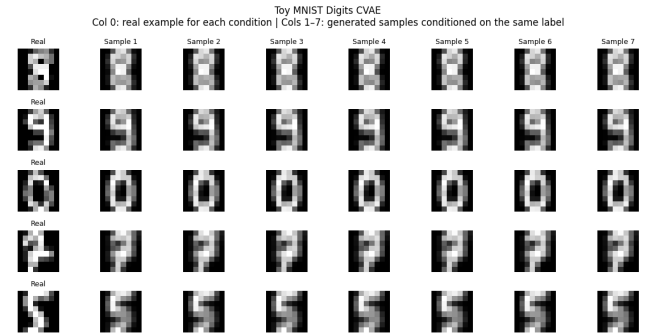
close to zero for both datasets, confirming that the approximate posterior effectively converged to a standard Gaussian distribution. In rare instances, the KL divergence stabilized around 1; even in these cases, the results indicate that the approximate posterior remained largely consistent with the desired Gaussian prior.

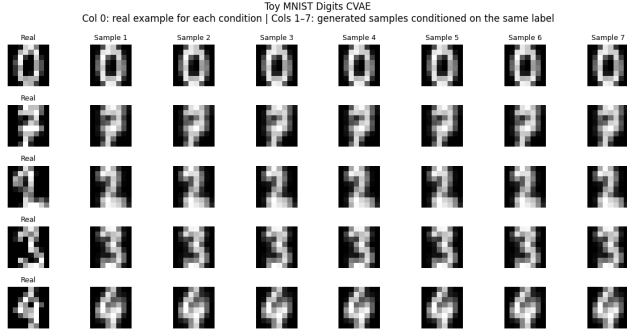
Wall-clock time was consistently less than 0.5 a second per epoch for both datasets. The wall-clock time was very dependent on the network parameters (h_{dim} and l_{dim}) as well as the batch size (B). However, the quality or reconstruction did not meaningfully increase given a change in these network hyperparameters or batch size.



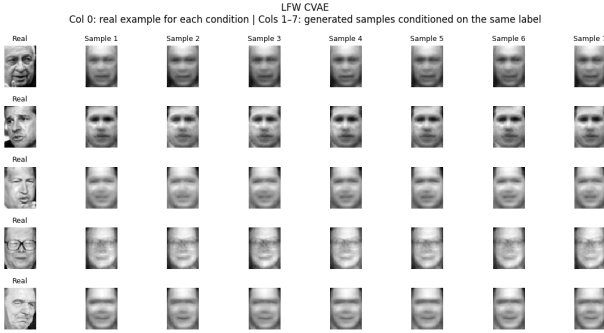
As mentioned, our implementation was compared against the official PyTorch / TensorFlow CVAE implementation and we found the results were very much comparable. The BCE and KL Divergence steady state losses were practically the same, with slightly different transient responses and an expectedly slightly faster wall-clock time.

Image Reconstruction

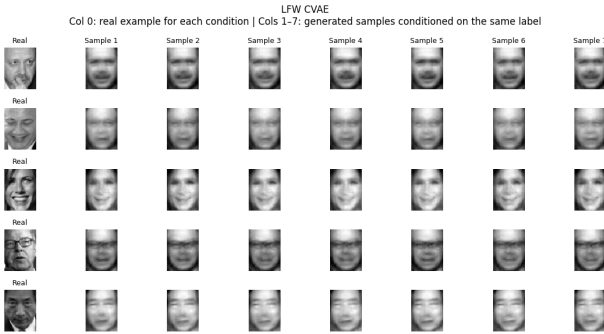




Toy MNIST Digits CVAE
Col 0: real example for each condition | Cols 1-7: generated samples conditioned on the same label



LFW CVAE
Col 0: real example for each condition | Cols 1-7: generated samples conditioned on the same label



LFW CVAE
Col 0: real example for each condition | Cols 1-7: generated samples conditioned on the same label

Conclusion

The implementation and analysis presented here reaffirm the effectiveness and adaptability of the Conditional Variational Autoencoder for state compression, representation learning, and conditional reconstruction. By rebuilding the CVAE architecture we verified both the mechanics and the mathematical foundations that underlie its performance. The results obtained across the MNIST Digits and LFW datasets demonstrate that the ability to reliably capture latent structure, converge toward a stable Gaussian prior, and produce reconstructions comparable to official PyTorch baselines.

The consistency of the BCE and KL Divergence behaviors, together with efficient computational performance, further highlights the practicality of CVAEs as tools for Autonomous Systems usage, particularly in environments where compact, condition-aware state representations are essential. While reconstruction quality is understandably bounded by architectural choices and dataset constraints; the broader objective of understanding and validating the CVAE framework was achieved.

Beyond reproducing prior results, this work also emphasizes the interpretability of the latent space and the value of conditioning as a mechanism for separating high-level information from lower-level information. This separation remains a powerful concept for vision based robotics, reinforcement learning, and any scenario where decisions must be made under uncertainty with limited computational resources.

Overall, the findings confirm that CVAEs remain a robust and versatile model class for future research, development, and deployment.

Contributions & Release

Contributions:

- Gabriel Agostine: CVAE implementation, Report

The authors do grant permission for this report to be posted publicly.

Our implementation successfully produced accurate reconstructions of the original states (images) for both datasets. Furthermore, we observed that the reconstruction quality was effectively equivalent to that of the reference PyTorch implementation, with any differences being negligible.

The reconstructed images exhibit a degree of blurriness, which is consistent with expectations. As in the original work by Sohn et al. 2015, [1], all images were downsampled to 50% of their original resolution prior to training. For the LFW dataset in particular, this effect is more visually apparent because the source images are displayed at full resolution, while the reconstructions are shown at the reduced resolution.

Additionally, the reconstructed LFW faces tend to appear oriented directly toward the viewer. This behavior is characteristic of CVAE models: enforcing a Gaussian prior on the latent distribution encourages the decoder to generate samples that gravitate toward an “average” representation for each individual or condition, producing front-facing features even when the original images exhibit pose variation.

References

- [1] Sohn, K., Lee, H., and Yan, X., “Learning Structured Output Representation using Deep Conditional Generative Models,” *Advances in Neural Information Processing Systems*, Vol. 28, edited by C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf.
- [2] Dykeman, I., “CVAE: Conditional Variational Auto-Encoder,” <https://ijdykeman.github.io/ml/2016/12/21/cvae.html>, Dec. 2016.
- [3] Josefsson, S., “The Base16, Base32, and Base64 Data Encodings,” RFC 4648 (Internet Standards Track) <https://www.rfc-editor.org/rfc/rfc4648>, Oct. 2006.
- [4] Abdelli, K., Griesser, H., Neumeyr, C., Hohenleitner, R., and Pachnicke, S., “Degradation Prediction of Semiconductor Lasers using Conditional Variational Autoencoder,” , 11 2022. <https://doi.org/10.48550/arXiv.2211.02847>.
- [5] Kingma, D. P., and Welling, M., “Auto-Encoding Variational Bayes,” , 2022. URL <https://arxiv.org/abs/1312.6114>.
- [6] Kullback, S., and Leibler, R. A., “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, Vol. 22, No. 1, 1951, pp. 79–86. URL <http://www.jstor.org/stable/2236703>.